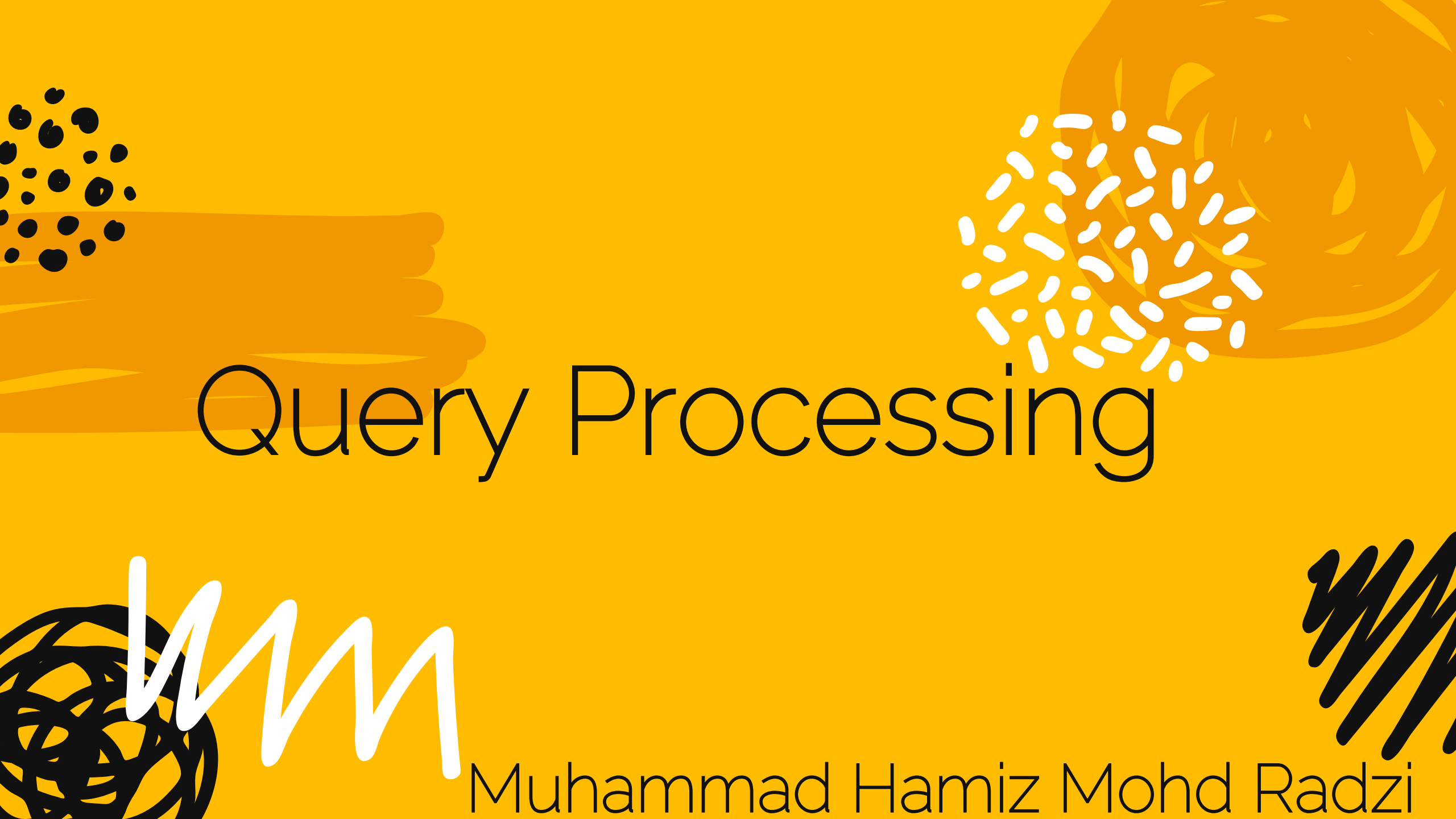




Query Processing

Muhammad Hamiz Mohd Radzi



Objectives

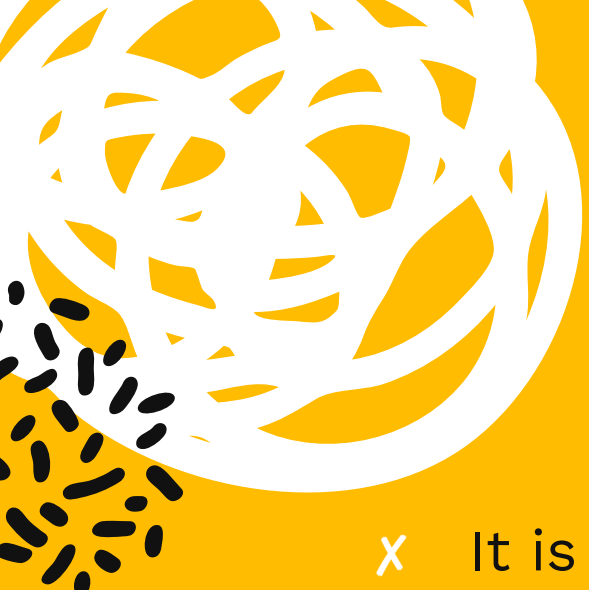
At the end of this lesson, you should be able to:

- x Define Query Processing, Query Optimization and Query Decomposition.
 - x Compare different strategies of Query Processing.
 - x Explain phases of Query Processing.
 - x Explain advantages and disadvantages of dynamic and static optimization.
 - x Describe the stages involve in Query Decomposition.
- 



Introduction

- x When the relational model was first launched commercially, one of the major criticisms often cited was **inadequate performance of queries**.
 - x Since then, a significant amount of research has been devoted to **developing highly efficient algorithms** for processing queries.
 - x There are many ways in which a complex query can be performed, and one of **the aims** of query processing is to **determine which one is the most cost effective**.
- 

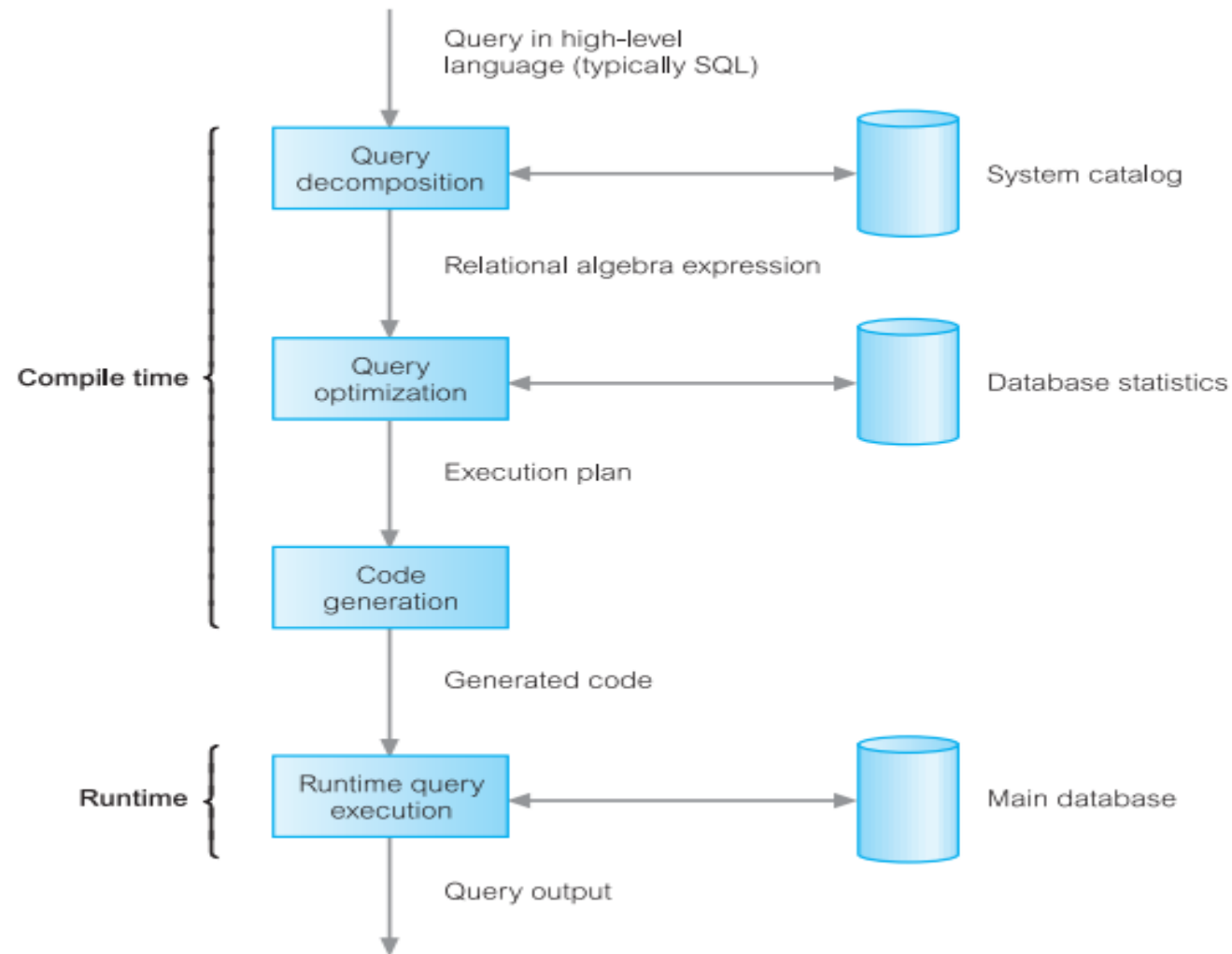
- 
- 
- 
- x It is **programmer's responsibility** to select the most appropriate execution strategy.
 - x SQL (declarative language) user specifies what data is required rather than how it is to be retrieved. (to make SQL more universally usable)
 - x Giving DBMS the responsibility for selecting best strategy and more control over system performance.



Overview of Query Processing

- x **QP** : Activities involved in **parsing, validating, optimizing and executing** a query.
- x **Aim of QP**: To **transform** a query written in a **high level language** (SQL) into a **correct and efficient execution strategy** expressed in a **low-level language** (implementing Relational Algebra) and to **execute** the strategy to retrieve required data.
- x An important aspect of query processing is **query optimization**.

Phases of Query Processing





Dynamic VS Static Processing

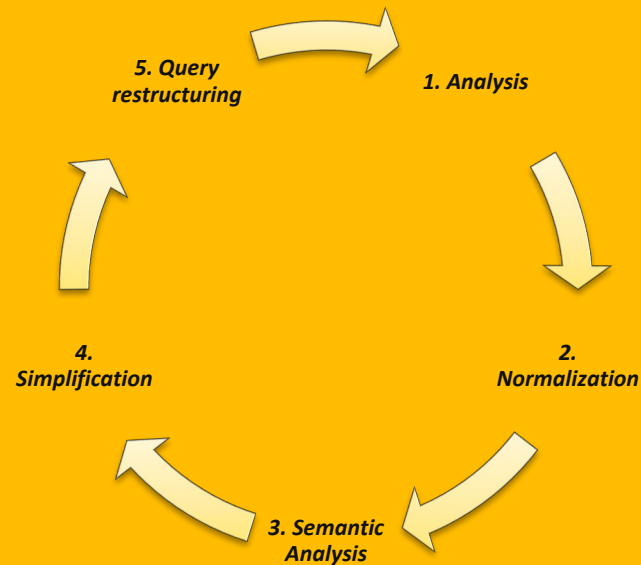
There are two choices for when the first three phases of query processing can be carried out.

- x 1. Dynamically carry out decomposition and optimization every time the query is run.
 - x 2. Just do it once (statically) for query to parsed, validated, and optimized
- 

	ADVANTAGES	DISADVANTAGES
DYNAMIC	All information required to select an optimum strategy is up to date.	Performance of the query is affected because the query has to be parsed, validated and optimized before it can be executed. May be necessary to reduce the number of execution strategies to be analyzed to achieve an acceptable overhead, which may have the effect of selecting a less optimum strategy.
STATIC	Runtime overhead is removed and more time available to evaluate a larger number of execution strategies (increasing the chances to find optimum strategy) For queries that are executed many times, taking some additional time to find a more optimum plan may prove to be highly beneficial.	Execution strategy is chosen to be optimal when the query is compiled may no longer be optimal when the query is running.

Query Decomposition

- x Query decomposition is the first phase of query processing.
- x The **aims** of query decomposition are to **transform a high-level query into a relational algebra query**, and **to check that the query is syntactically and semantically correct**.
- x The typical stages of query decomposition are





Analysis



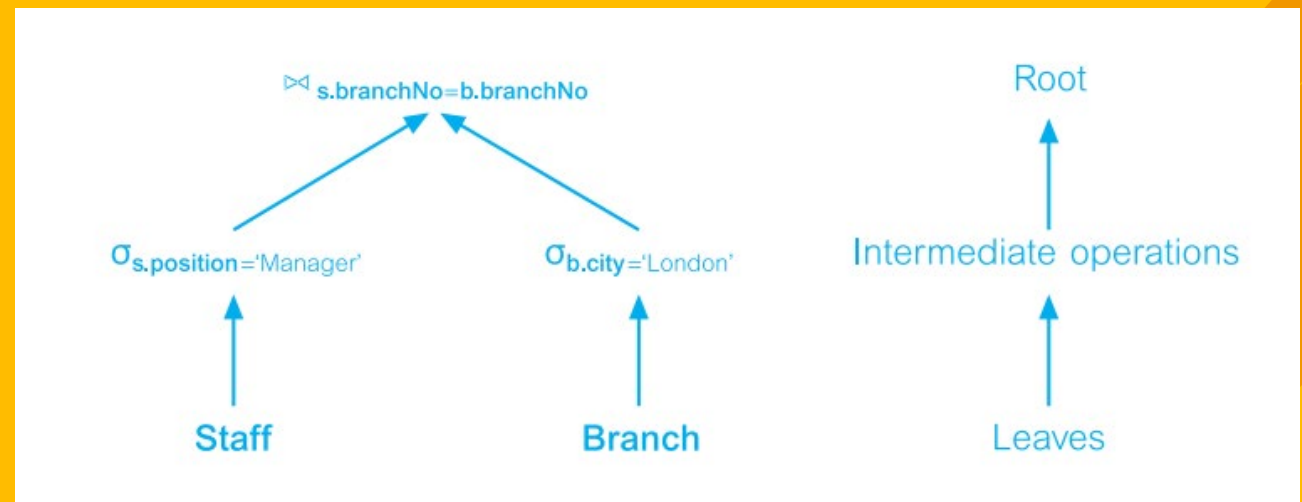
- X The query is **lexically and syntactically analyzed** using the techniques of programming language compilers.
 - X Verifies that the **relations and attributes** specified in the query **are defined in the system catalog**.
 - X Verifies that any **operations applied** to database objects **are appropriate** for the object type.
 - X On completion of the analysis, the **high-level query** has been **transformed** into some **internal representation** (query tree) that is more suitable for processing.
- 

Analysis - Example

```
SELECT staffNumber  
FROM Staff  
WHERE position > 10;
```

This query would be rejected on two grounds:

- (1) In the select list, the attribute staffNumber is not defined for the Staff relation (should be staffNo).
- (2) In the WHERE clause, the comparison '>10' is incompatible with the data type position, which is a variable character string.



Normalization

To transform the query into normalized form to facilitate further processing.

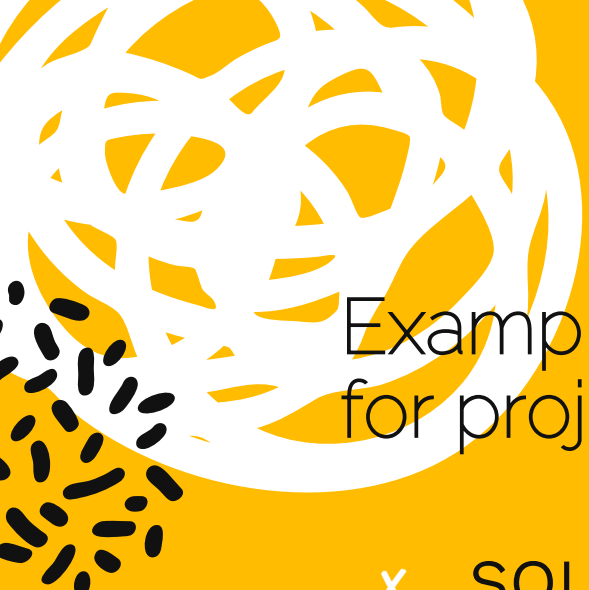
- x Transformation rules are:
 - x Conjunctive NF: A sequence of conjuncts that are **connected with the $\wedge$ (AND) operator**. Each conjunct contains one or more terms connected by the $\vee$ (OR) operator.
 - x Disjunctive NF: A sequence of disjuncts that are connected with **the $\vee$ (OR) operator**. Each disjunct contains one or more terms connected by the $\wedge$ (AND) operator.

- Conjunctive normal form

$$(p_{11} \vee p_{12} \vee \cdots \vee p_{1n}) \wedge \cdots \wedge (p_{m1} \vee p_{m2} \vee \cdots \vee p_{mn})$$

- Disjunctive normal form

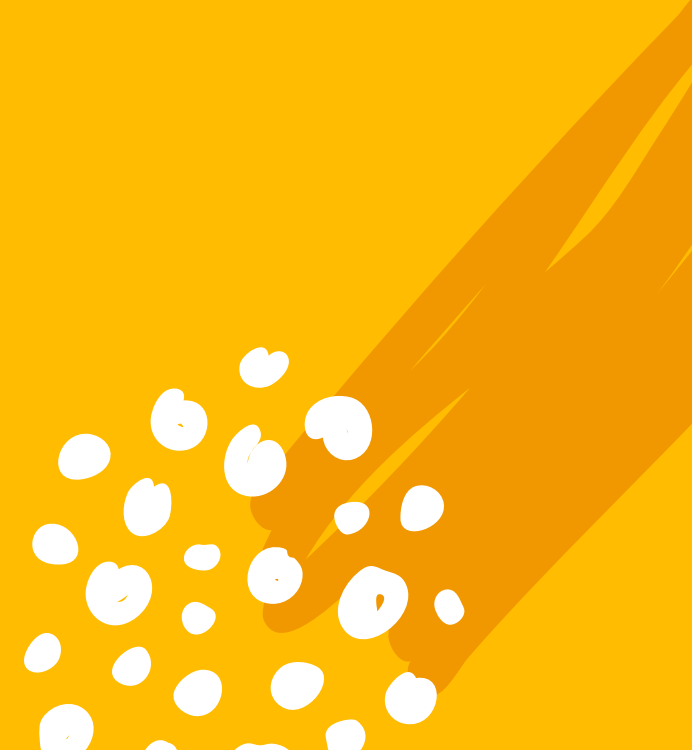
$$(p_{11} \wedge p_{12} \wedge \cdots \wedge p_{1n}) \vee \cdots \vee (p_{m1} \wedge p_{m2} \wedge \cdots \wedge p_{mn})$$

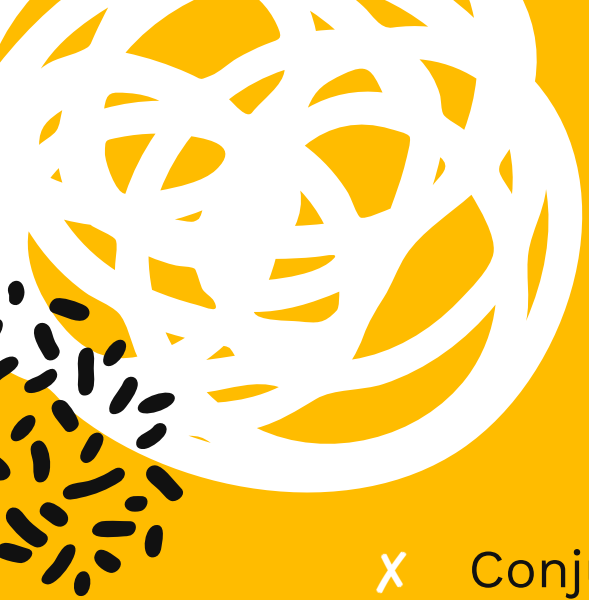


Example: Find an employees who have been working for project P1 for 12 or 24 months.

x SQL :

```
SELECT e.empName  
FROM emp e, assignment a  
WHERE e.empNo = s.empNo  
AND a.projNo = 'P1'  
AND duration = 12 OR duration = 24;
```





X Conjunctive:

$(e.empNo = a.empNo) \wedge (a.projNo = 'P1') \wedge (duration = 12 \vee duration = 24)$

X Disjunctive:

**$(e.empNo = a.empNo \wedge a.projNo = 'P1' \wedge duration = 12) \vee$
 $(e.empNo = a.empNo \wedge a.projNo = 'P1' \wedge duration = 24)$**





Semantic Analysis



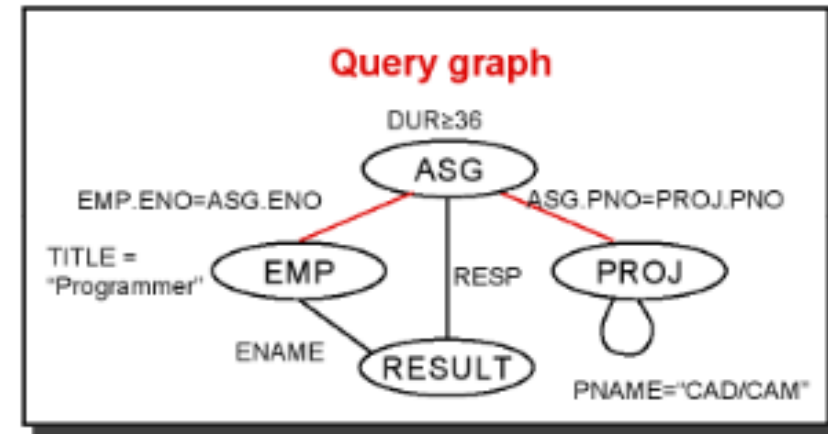
- x Identify and reject **type incorrect** or **semantically incorrect** queries.
 - x Type incorrect: attributes and relations name **not in global schema** and **data type of attribute is not match**.
 - x Semantically incorrect: the relations are **not joined** in the **query and join graphs**.
- 

- **Example:** Consider a query:

```

SELECT  ENAME, RESP
FROM    EMP, ASG, PROJ
WHERE   EMP.ENO = ASG.ENO
AND     ASG.PNO = PROJ.PNO
AND     PNAME = "CAD/CAM"
AND     DUR ≥ 36
AND     TITLE = "Programmer"

```

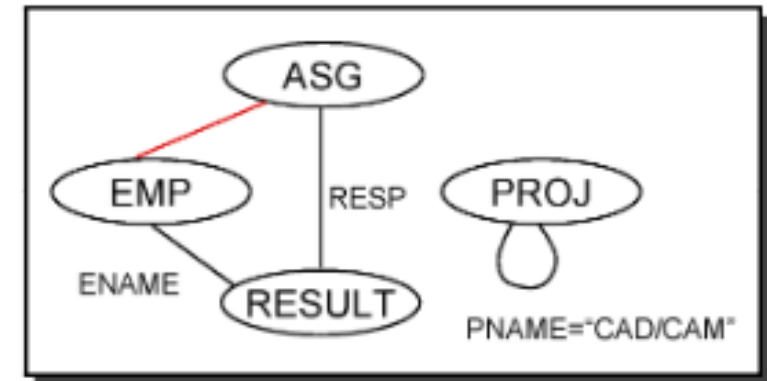


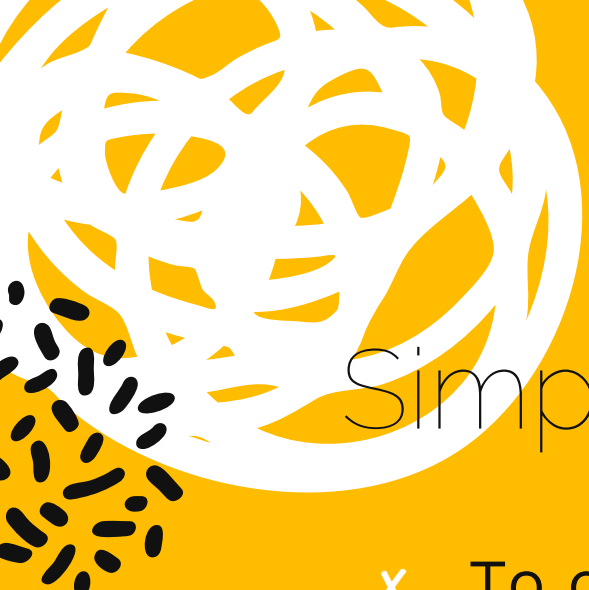
- **Example:** Consider the following query and its query graph:

```

SELECT  ENAME, RESP
FROM    EMP, ASG, PROJ
WHERE   EMP.ENO = ASG.ENO
AND     PNAME = "CAD/CAM"
AND     DUR ≥ 36
AND     TITLE = "Programmer"

```





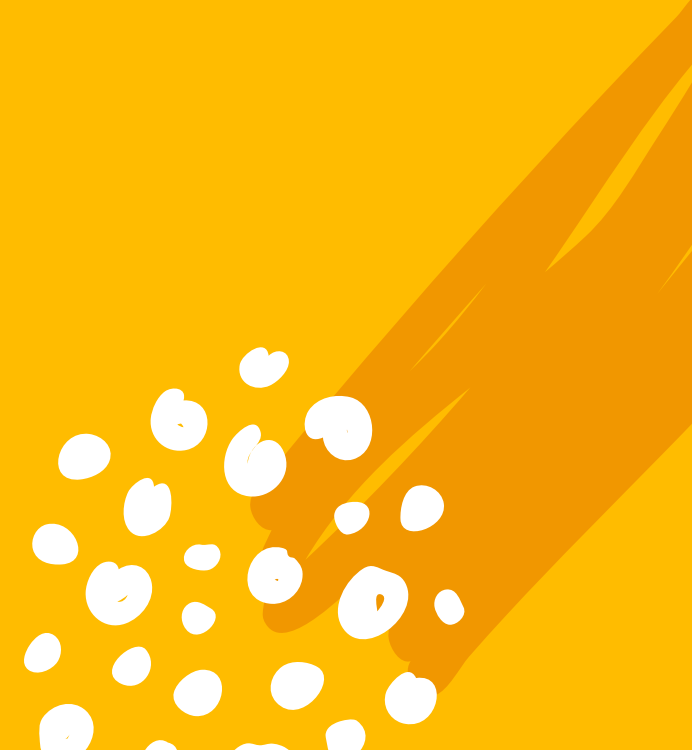
Simplification



- x To detect **redundant qualifications**, **eliminate common sub-expressions** , and **transform** the query to a **semantically equivalent** but more easily and efficiently computed form.
 - x Access restrictions, view definitions, and integrity constraints are considered at this stage.
- 

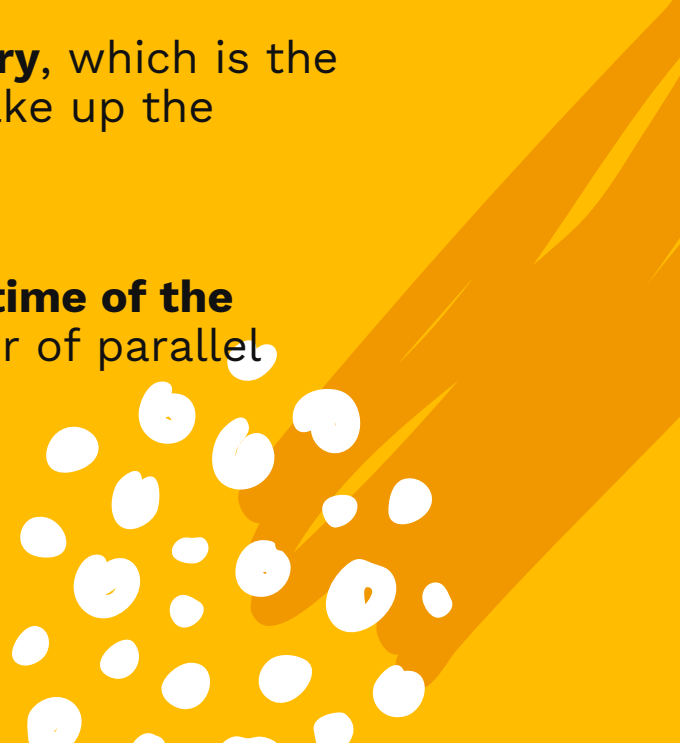


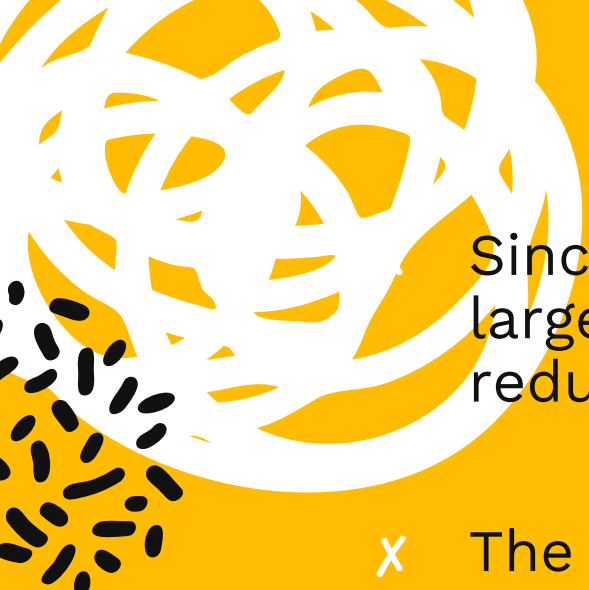
Query Restructuring

- x In the final stage of query decomposition, the query is restructured to provide a more efficient implementation.
- 
- 



Query Optimization (QO)

- X QO: The activity of **choosing an efficient execution strategy** for processing a query.
 - X Generally, we try to reduce the **total execution time of the query**, which is the sum of the execution times of all individual operations that make up the query.
 - X However, **resource usage** may also be viewed as the **response time of the query**, in which case we concentrate on maximizing the number of parallel operations.
- 



Since the problem is **computationally hard to control** with a large number of relations, the strategy adopted is generally reduced to finding a **near optimum solution**.

- x The first technique of QO uses **heuristic rules** that order the operations in a query.
 - x The other technique **compares different strategies** based on their relative costs and selects the one that minimizes resource usage.
 - x Both methods of query optimization depend on **database statistics** to evaluate properly the different options that are available.
- 



The **accuracy and currency** of these statistics have a significant bearing on the **efficiency** of the execution strategy chosen.

- x The statistics cover information about relations, attributes, and indexes.
 - x There a lot of methods in order to keep the statistics which is update regularly (problematic), periodic update or user specified.
- 

Find all Managers who work at a London branch.

We can write this query in SQL as:

```
SELECT *  
FROM Staff s, Branch b  
WHERE s.branchNo = b.branchNo AND  
      (s.position = 'Manager' AND b.city = 'London');
```

Three equivalent relational algebra queries corresponding to this SQL statement are:

- (1) $\sigma_{(\text{position}='Manager') \wedge (\text{city}='London') \wedge (\text{Staff.branchNo}=\text{Branch.branchNo})}(\text{Staff} \times \text{Branch})$
- (2) $\sigma_{(\text{position}='Manager') \wedge (\text{city}='London')}(\text{Staff} \bowtie_{\text{Staff.branchNo}=\text{Branch.branchNo}} \text{Branch})$
- (3) $(\sigma_{\text{position}='Manager'}(\text{Staff})) \bowtie_{\text{Staff.branchNo}=\text{Branch.branchNo}} (\sigma_{\text{city}='London'}(\text{Branch}))$

Calculation Result

x 1)

$$(1000 + 50) + 2 * (1000 * 50) = 101\ 050 \text{ disk accesses}$$

x 2)

$$2 * 1000 + (1000 + 50) = 3050 \text{ disk accesses}$$

x 3)

$$1000 + 2 * 50 + 5 + (50 + 5) = 1160 \text{ disk accesses}$$



Generated Code

- x The relational algebra got from QO is then transformed into SQL back

Runtime Query

- x The SQL generated is then tested into the physical database to see its performance
- 



References

- x Thomas Connolly and Carolyn Begg, Database Systems: A Practical Approach to Design, Implementation, and Management, 6th Edition, Pearson, 2015, ISBN: 978-01329432
- 